

ENGINEERING CASE STUDY · 2026

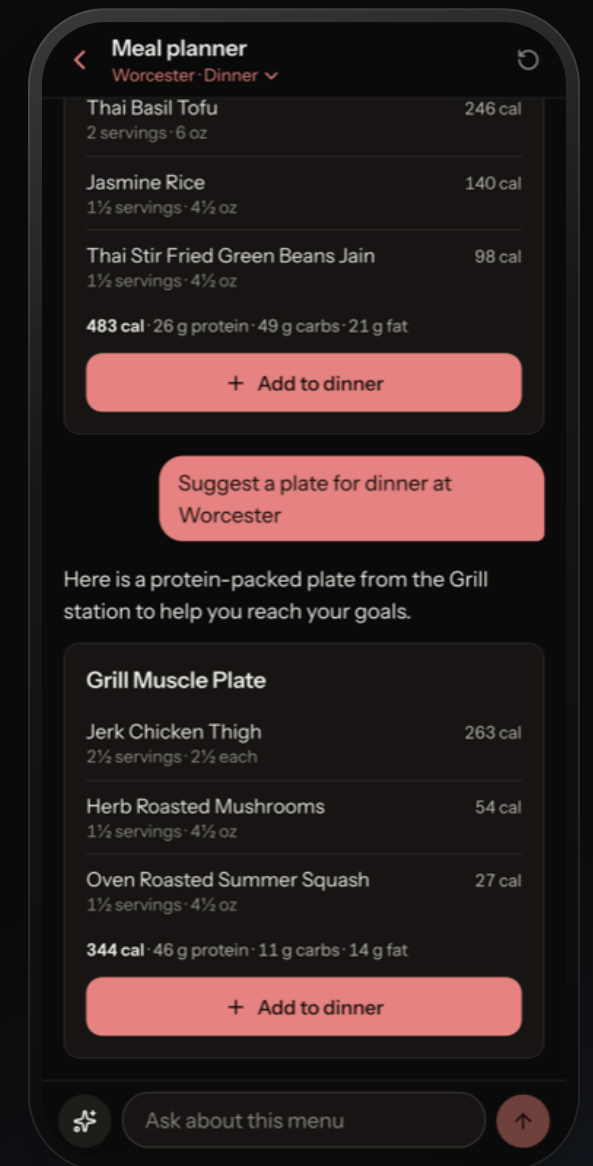
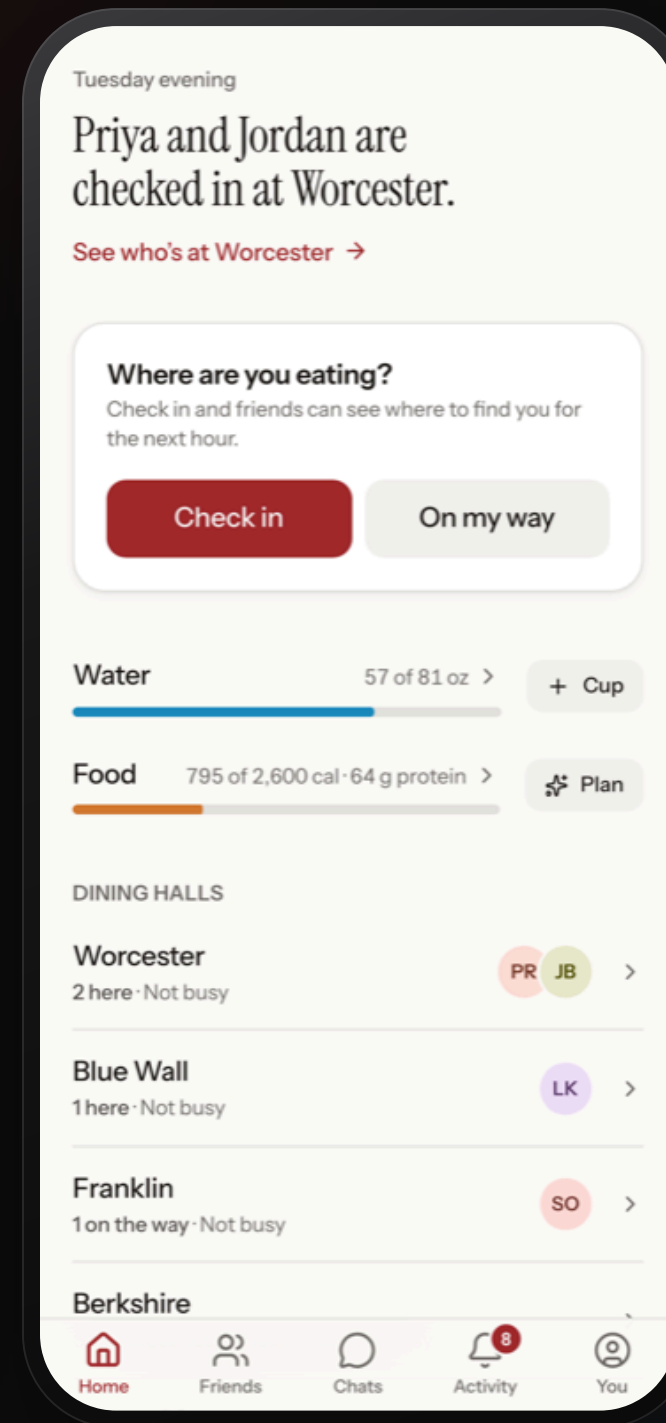
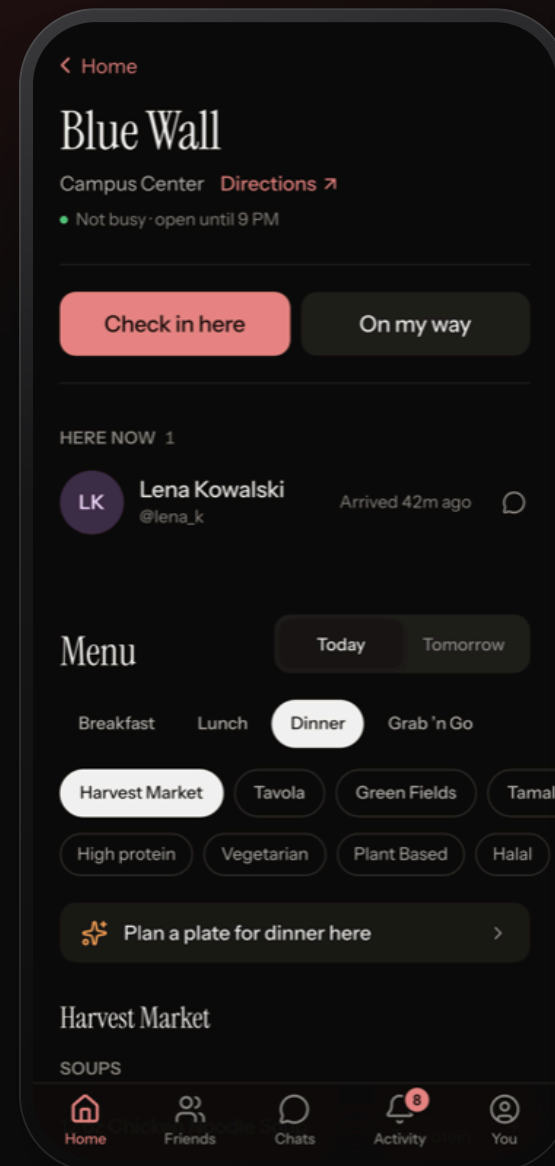
# DineSight

A live dining app for UMass Amherst: where your friends are eating, what's on the menu tonight, and what to put on your plate.

Muhammad Mahad Mushtaq

dine-sight.vercel.app

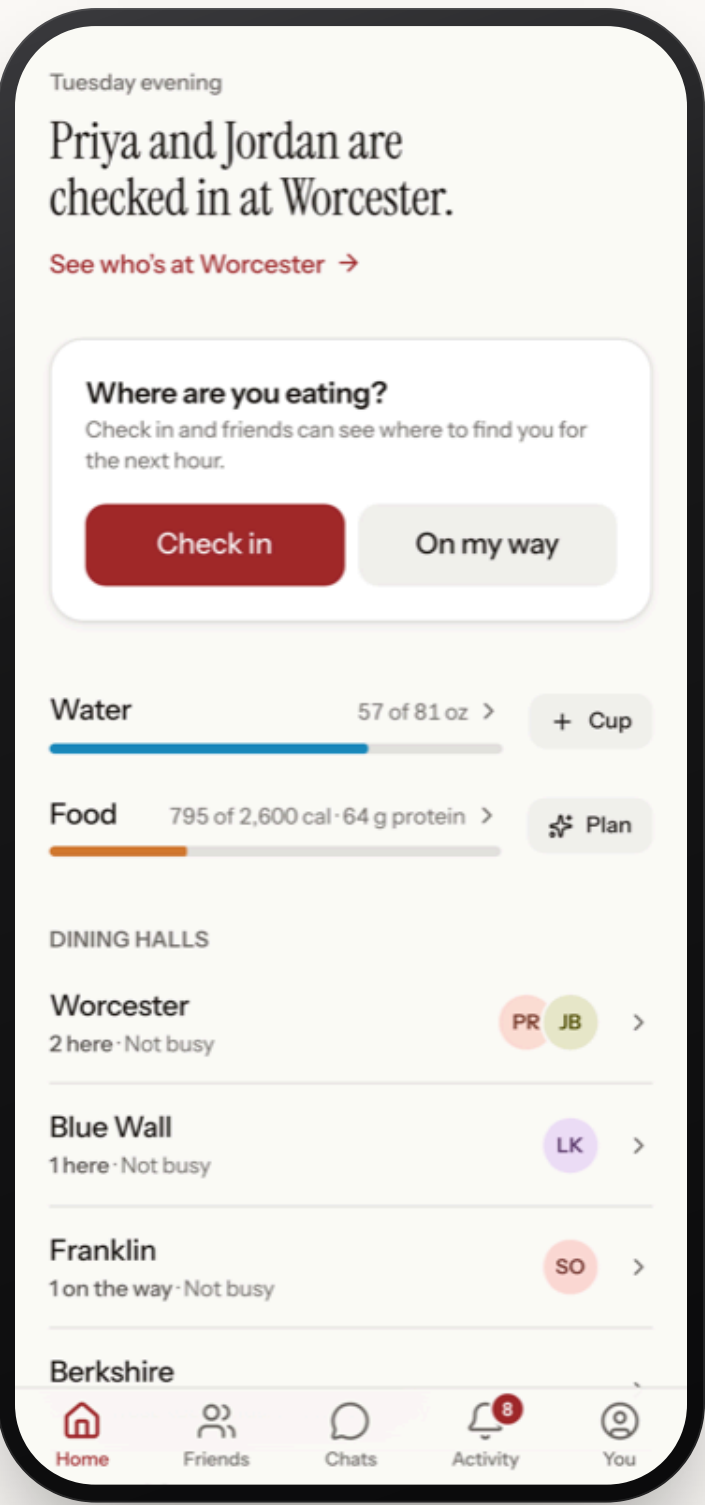
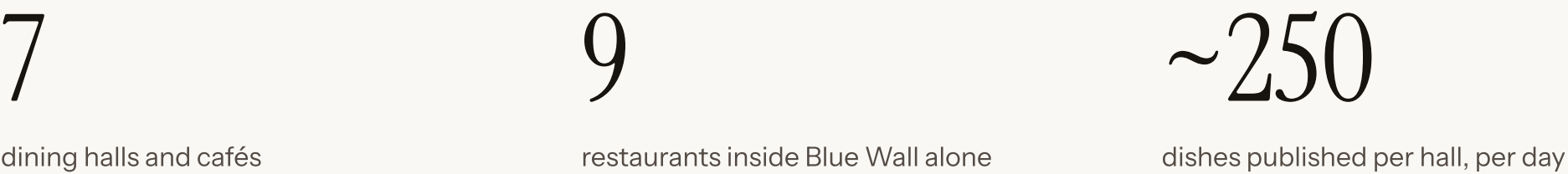
github.com/mahad1921/DineSight



# Seven dining halls. No idea who's there.

UMass runs the largest campus dining program in the United States. Students still text “where are you eating?” into group chats, walk across campus to a hall that turns out to be packed, and swipe in before they know what's on the line.

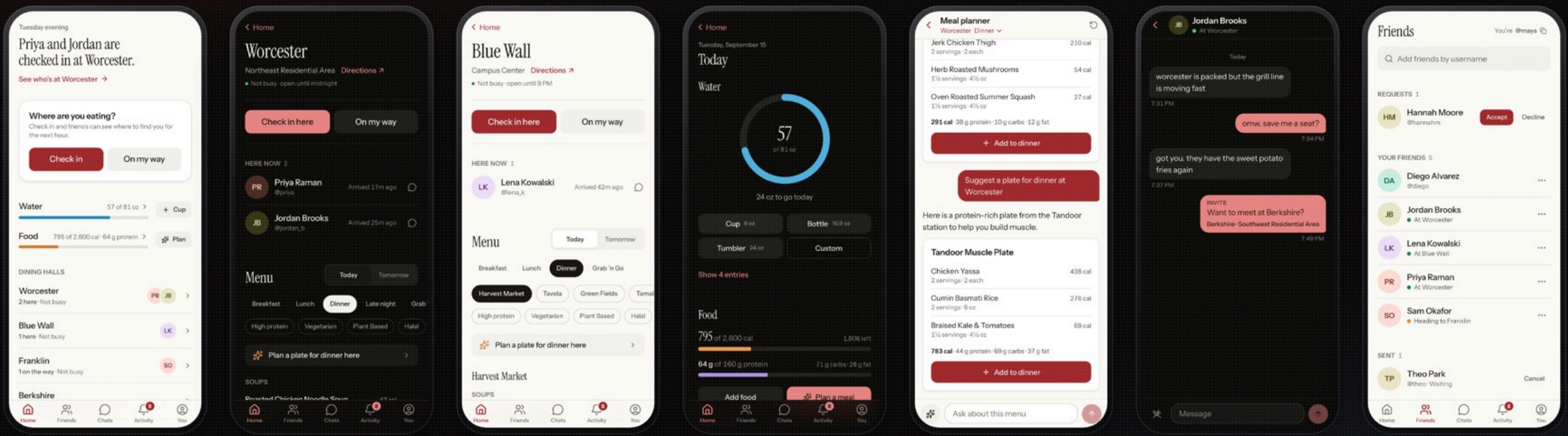
The information already exists. UMass Dining publishes menus, nutrition, hours and live busyness — just nowhere useful at the moment you're deciding.



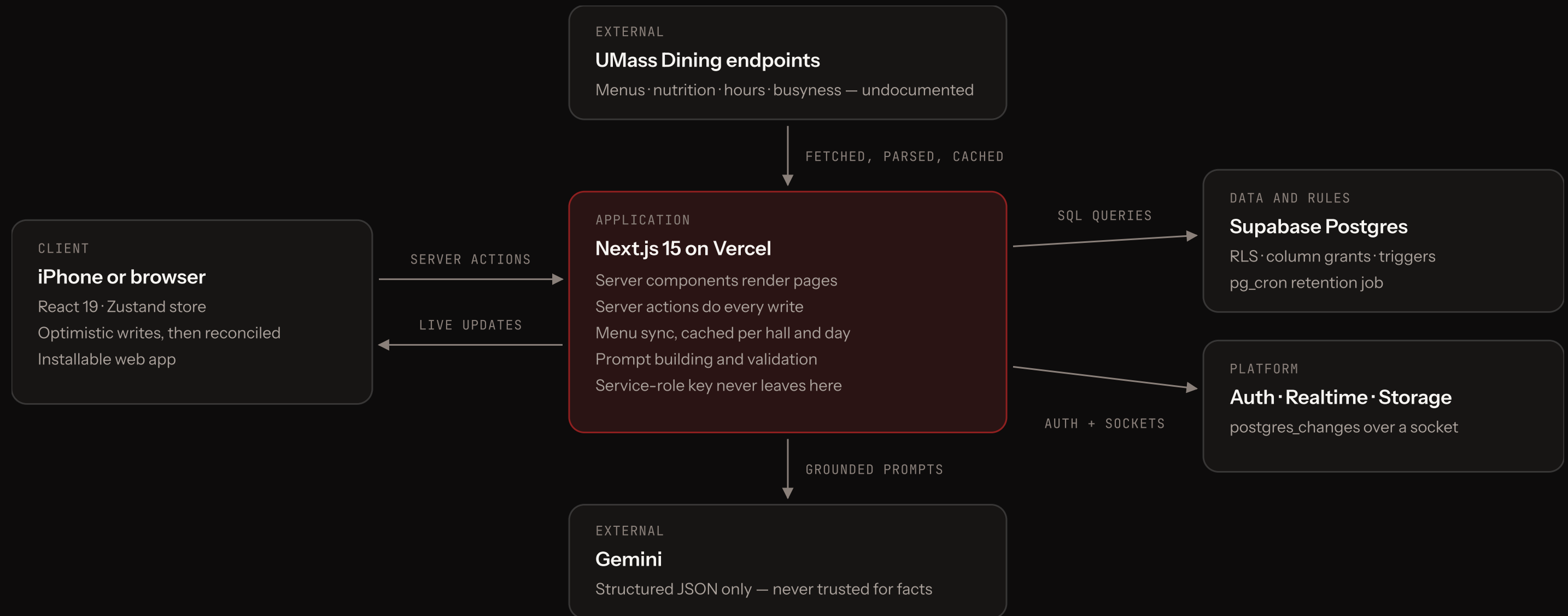
# Four things, done properly.

One system, light and dark.

Seven of the app's screens, from a shared set of design tokens.



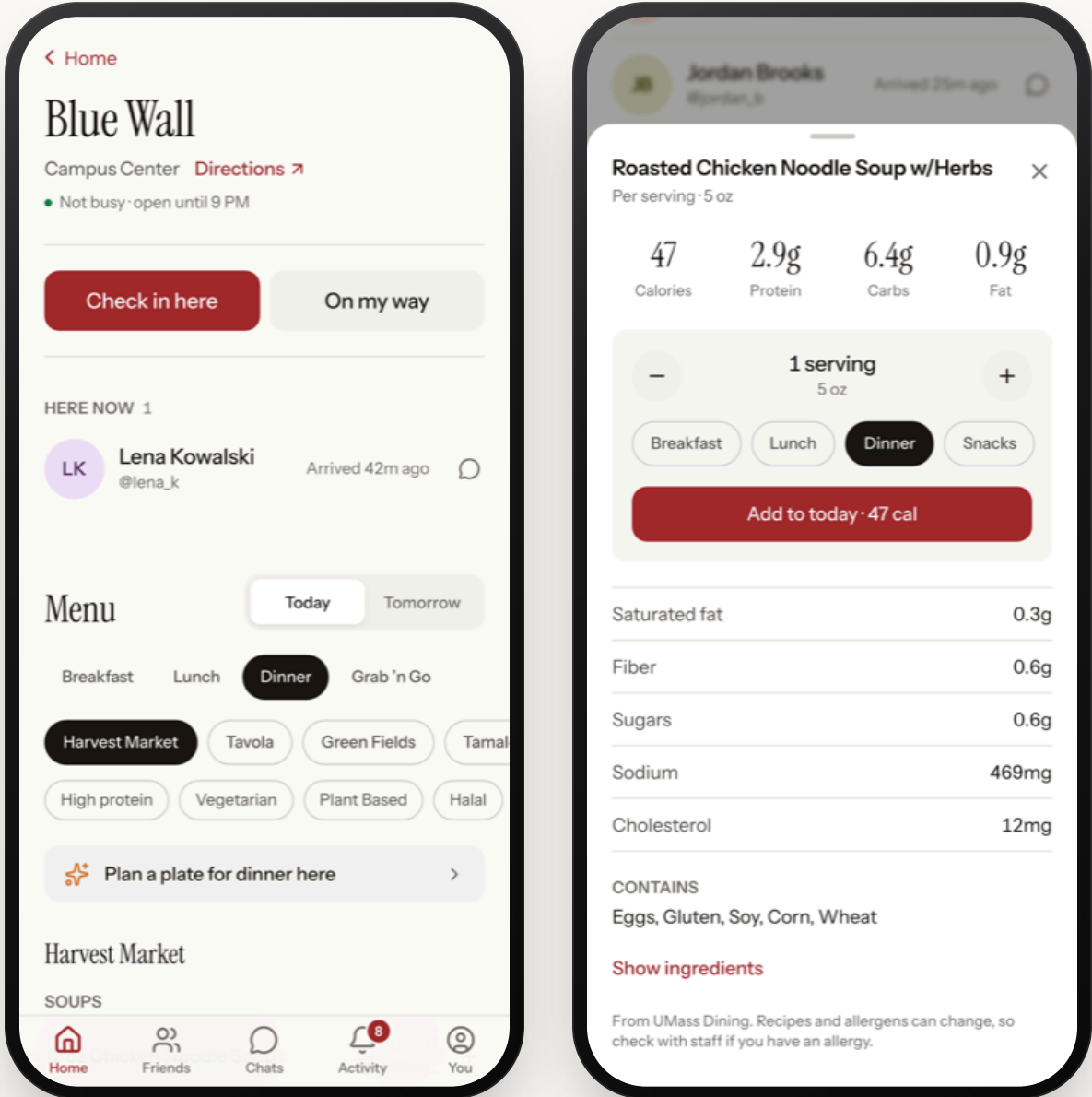
# One Next.js app, one Postgres, no backend to babysit.



# Building on somebody else's undocumented API.

UMass Dining's app talks to endpoints that aren't published, versioned or supported. DineSight treats them as hostile input and never lets them block a page.

- **Reverse-engineered, then documented.** Two endpoints: a location feed with hours and busyness thresholds, and an HTML fragment per hall and day whose nutrition hides in `data-*` attributes.
- **Parsed defensively.** Malformed dishes are skipped, never thrown. FoodPro's codes are cleaned up: "12 OZL" becomes "12 fl oz", "Brkfst" becomes "Breakfast".
- **Cached per hall and day.** One request claims the sync through a Postgres advisory-style claim; everyone else reads the cache. Today's menu is refreshed after 4 hours, tomorrow's after 12, past days never.
- **Degrades honestly.** If a fetch fails, the previous menu stays. If a sync is mid-flight, the client shows a loading state and retries instead of "nothing here".



# The database decides. The client only asks.

Every rule lives in Postgres, so a forged request from a modified client fails in the same place a UI bug would.

- **Row-level security everywhere.** Check-ins are visible to accepted friends only; food logs and planner conversations to nobody but their owner.
- **Per-column grants.** Clients can insert a check-in but not choose its expiry; they can update a display name but never an email or a `user_id`.
- **Triggers own the truth.** Timestamps, campus dates and auto-checkout windows are stamped server-side. A client can't backdate a food log or read-receipt.
- **Quotas the client can't reset.** AI usage is counted in a table with no client grants at all, claimed through a `SECURITY DEFINER` function that takes no limit argument.

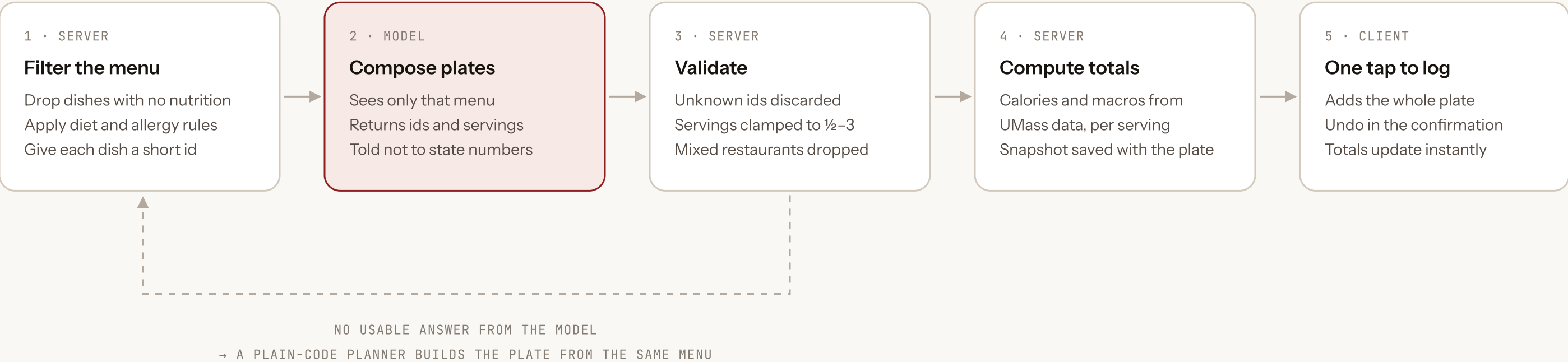
```
-- 72 database tests assert what is refused, not just what works
SELECT throws_ok(
  $$ UPDATE friendships SET status = 'accepted' $$,
  '42501', NULL,
  'the requester cannot accept their own request'
);

SELECT is(
  (SELECT count(*)::int FROM food_logs),
  0,
  'friends cannot see your food log'
);
```

Run with `supabase test db` against a real Postgres, as the same roles the app uses.

# A meal planner that cannot invent food.

A language model is good at composing a plate and bad at arithmetic, and it will happily invent a dish that sounds plausible. So it never gets the last word on anything factual.



# Grounding beats prompting.

## The bug that changed the design

Asked “is anything here gluten free?”, the model named a soup that UMass lists as containing gluten and wheat. Plates were already validated; free text wasn't.

**Fix →** allergies and diets mentioned anywhere in the conversation are now applied in code. Conflicting dishes never reach the model, so it cannot name one. Re-tested against live Gemini: correct.

## Adversarial checks, run against the real model

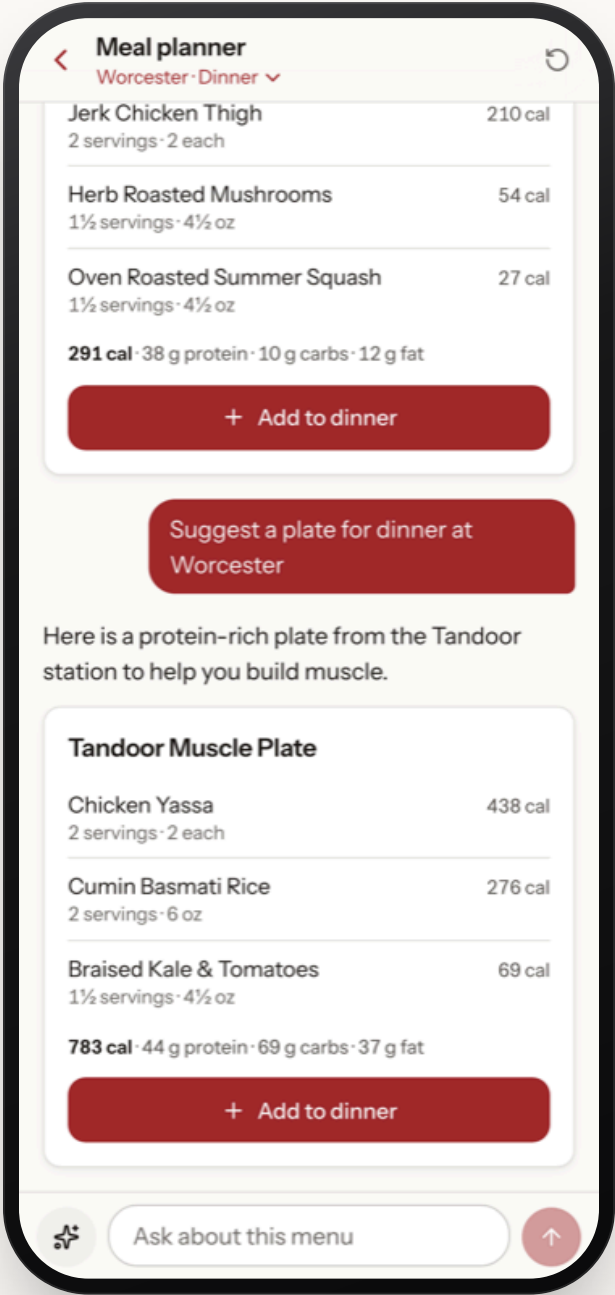
Crash diet (“600 calories a day”) · “ignore your rules” · allergy mentioned mid-chat · a crisis message · off-topic requests.

**Result →** declined, declined, filtered, fixed crisis response with 988 and campus counseling, declined. Each is a test, not a hope.

## Cost and privacy

Free-tier Gemini, shared across everyone using the app.

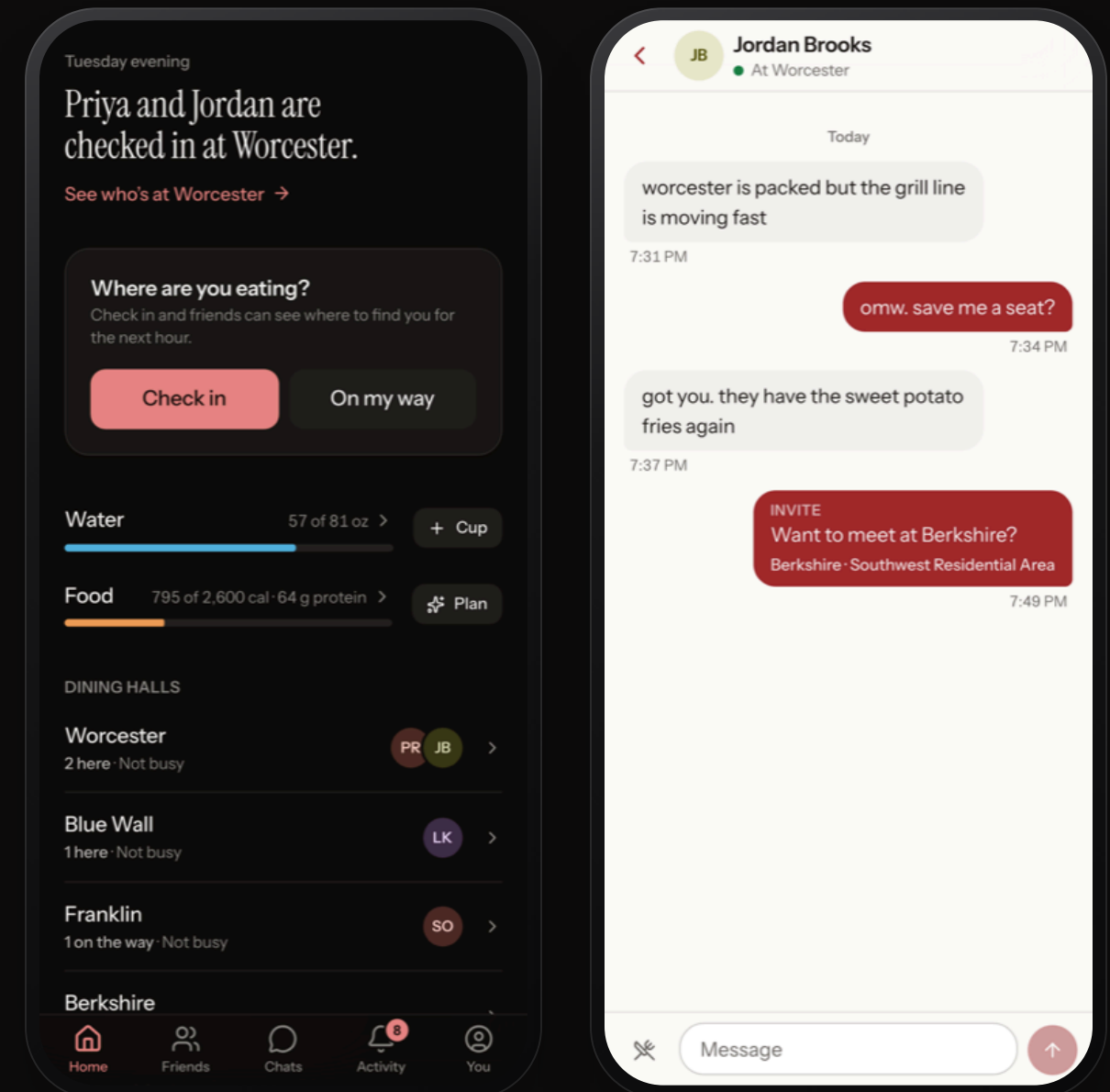
**Design →** 30 requests per person per 12 hours, enforced in Postgres. Prompts carry the menu, the goal and today's totals — never names or accounts. Conversations delete themselves after 12 hours.



# Instant, then correct.

Every write shows its result before the network answers, and reconciles when it does.

- **Optimistic with a version guard.** Each check-in mutation takes a version; a slow response can't overwrite a newer action, and a failed one rolls back with an explanation.
- **Realtime, authenticated.** Check-ins, messages and notifications arrive over `postgres_changes`. The socket is authenticated before subscribing — otherwise it silently joins anonymous and row-level security filters everything away.
- **Offline is a state, not a crash.** Going offline rolls the action back and says so; reconnecting catches up with one refetch.
- **Sessions end everywhere.** Signing out in one tab sends the others back to sign-in, returning to the page they were on.
- **Midnight is handled.** Totals roll over on the campus calendar, not the device's, including across daylight saving.



# Calorie tracking, built carefully.

Tracking food among college students can feed disordered eating. The constraints below are product decisions, enforced in code.

- **Off by default**, with a “just track” mode that has no targets at all.
- **A floor in the database.** No target below 1,200 kcal can be saved, by anyone, ever — not a client-side check.
- **Nothing sensitive leaves the phone.** The calculator's height, weight and age stay on the device; only the resulting targets are stored.
- **Neutral language.** Passing a target reads “Target reached”, not a red warning.
- **Private by construction.** No friend policy exists on food tables, so there is no setting that could leak them.

**Water is the opposite call:** the challenge is opt-in on both sides, shares daily totals only, and shows nothing until you share too.



# 416 tests, weighted where the risk is.

288

**Unit and component**

Vitest and Testing Library: parsing, portion maths, target calculator, prompt building, response validation, React screens.

72

**Database**

pgTAP against real Postgres, as the real roles. Most assert that something is *refused*.

56

**End to end**

Playwright against a real Supabase stack, on desktop Chrome, Pixel 7 and iPhone WebKit.

0

**Accessibility violations**

axe-core on every screen, light and dark, at 320 px and up — part of the suite, not a one-off audit.

**What the tests are allowed to touch**

End-to-end tests sign up real users against a local Supabase, exercise realtime between two browser contexts, and run the AI path with the model mocked, so a rate limit can never fail the build. The live model is exercised separately, by hand, against a scripted set of adversarial prompts.

**A test-infrastructure bug worth naming**

Reload-after-save tests were flaky, worst on WebKit. The cause: Playwright's `waitForLoadState("networkidle")` resolves immediately on an already-loaded page, so reloads were cancelling the save they were meant to wait for. Replaced with an explicit wait for the server action's response; two full suite runs, clean.

# Found before users did.

A dedicated QA pass before launch, then again before each feature merge. Every fix below ships with a test.

## Blue Wall merged nine restaurants into one menu

“Add-ons” held 81 dishes; “Fountain Drink” appeared seven times. The food court reuses generic station names.

**Fix →** restaurant stored as its own column and browsed restaurant-first, the way UMass Dining lists it.

## Confirmation emails failed on a second device

Opening the link on a phone after signing up on a laptop showed “link expired”, even though the account was confirmed.

**Fix →** the callback detects the missing PKCE verifier and shows “You're all set”.

## Typical iPhone photos were rejected as avatars

A 2 MB limit was checked against the original file; normal phone photos are 3–5 MB.

**Fix →** resize to a 512 px JPEG in the browser first, which also converts HEIC on Safari.

## Sign-in flashed “couldn’t reach DineSight”

Next.js signals redirects by throwing; the catch block treated that as a network failure.

**Fix →** rethrow control-flow errors before handling real ones.

## Signing out in one tab left the others half-broken

Server-side sign-out only clears the cookie, so other tabs failed one action at a time instead of noticing.

**Fix →** tabs re-check the session on focus and return to sign-in, remembering the page you were on.

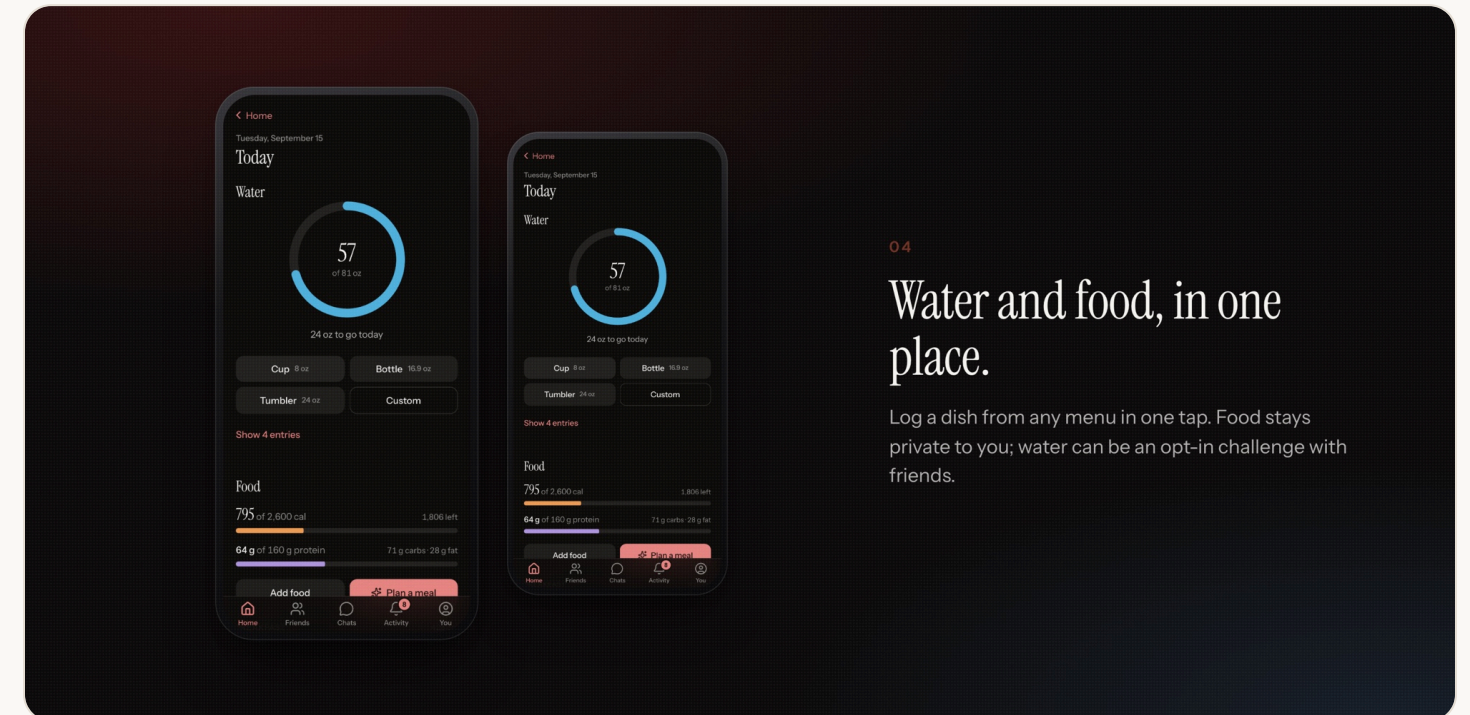
## Totals didn’t reset at midnight

A tab left open overnight kept yesterday's water and food, because “today” was fixed when the page loaded.

**Fix →** the clock tick notices a new campus day and refetches — tested across a daylight-saving boundary.

# One design system, both themes, one hand.

- **Tokens, not values.** Colour, type scale, radii and motion live in one place; light and dark ship together.
- **Built for a dining hall doorway.** Thumb-height controls, instant feedback, and an undo on everything that writes.
- **Details that only show on a phone.** Safe-area padding, no double-tap zoom, pressed states on rows, auto-capitalised names, a translucent status bar when installed.
- **Contrast held to AA** in both themes, including the two tokens that failed the first audit.
- **Link previews and an app icon** generated at the edge, so a texted link looks like a product.



# What it's made of, and how it ships.

|          |                                                                                    |
|----------|------------------------------------------------------------------------------------|
| FRONTEND | Next.js 15 App Router · React 19 · TypeScript · Tailwind CSS 4 · Base UI · Zustand |
| BACKEND  | Supabase Postgres · server actions · SQL migrations as the source of truth         |
| AI       | Gemini with structured output, schema validation and a deterministic fallback      |
| TESTING  | Vitest · Testing Library · pgTAP · Playwright · axe-core                           |
| DELIVERY | Vercel; Supabase's GitHub integration runs migrations on merge                     |

## Operational choices

- **Migrations are the schema.** Twelve numbered files; nothing is changed by hand in a dashboard.
- **Secrets stay server-side.** The service-role key is used only by the menu sync, never shipped to a browser.
- **Retention runs itself.** An hourly `pg_cron` job deletes planner conversations older than 12 hours; row-level security hides them immediately.
- **Documented as it was built.** A 21-chapter engineering journal records decisions, trade-offs and a ledger of every bug.

# Known gaps, in priority order.

Shipped software has a to-do list. These are mine, with the reasoning.

1

**Own email delivery**

Supabase's built-in sender only reaches project members and rate-limits hard. A dedicated provider restores confirmations and magic links for everyone.

2

**Account deletion and password change**

Expected in any real product, and a prerequisite for putting this in front of strangers rather than friends.

3

**Weekly trends**

Daily totals encourage fixation; a seven-day average is both healthier and more useful, and the data is already stored.

4

**Permission from UMass Dining**

The menu endpoints are public but undocumented. Anything beyond a campus project needs a conversation first.

5

**Push notifications**

“Two friends just checked in at Worcester” is the app's best moment, and it currently only exists if you're looking.

6

**A native shell**

Only needed for Apple Health and reliable notifications on iOS — not before.

THANK YOU

# Built, tested and documented end to end.

LIVE APP

[dine-sight.vercel.app](https://dine-sight.vercel.app)

SOURCE

[github.com/mahad1921/DineSight](https://github.com/mahad1921/DineSight)

JOURNAL

ENGINEERING\_JOURNAL.md — 21 chapters, full bug ledger

Menus, nutrition and busyness data belong to UMass Dining. Screenshots use demo accounts, not real students.